CostScale

ML-driven hybrid container and serverless resource management on AWS

Anamay Narkar (16010123040)
Ananya Dhiman (16010123041)
Anoushka Nelson (16010123049)

B.Tech Computer Engineering, K.J. Somaiya School of Engineering, Mumbai

[TO FILL: course name] | October 2026

The problem

Predict workload, then allocate resources

- Containers / VMs: cheap per unit, but slow to provision
- Lambda: instant, pay per use, several times costlier per unit
- Load varies; most functions are idle, a few are very busy
- Goal: lowest cost while serving every request in time

101

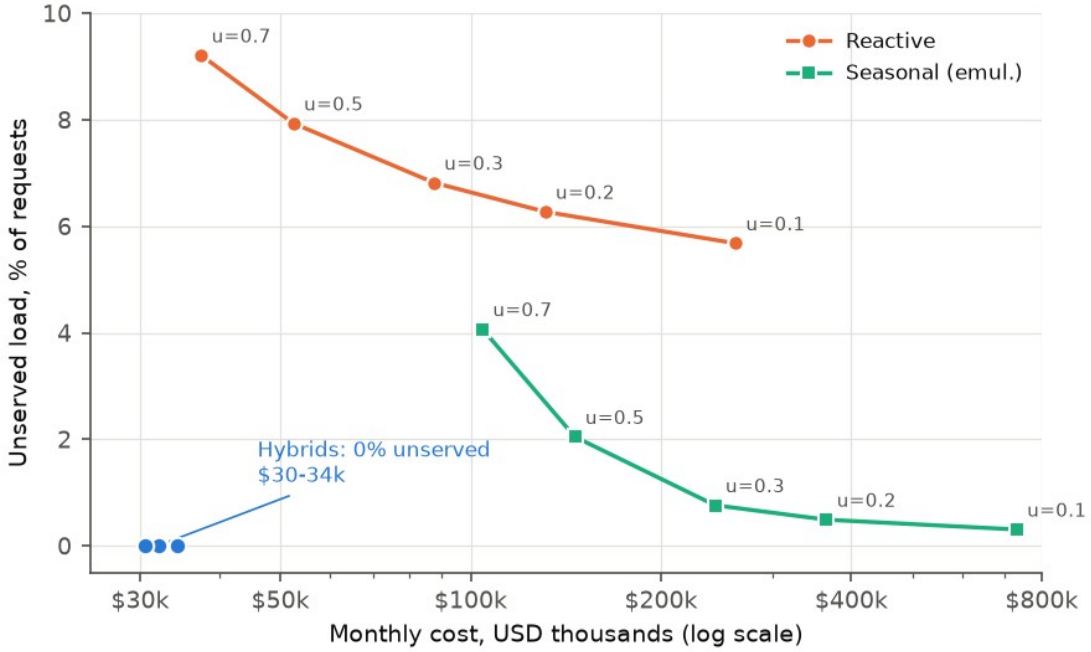
heavy Azure functions simulated
(>= $50/month on training days)

28,859

functions present on all 14 days
(of 72k in the trace)

Why plain autoscaling is not enough

Every reactive setting drops load or overpays



- Reactive u=0.7: $37.5k, 9.2% load dropped
- Reactive u=0.3: $87.5k, still 6.8% dropped
- Hybrid-rolling: $32.2k, 0% dropped
- Test days, 101 heavy functions, AWS Mumbai prices

The idea: a base plus overflow

Newsvendor: base = q-quantile of forecast load*

- Base: EC2 Auto Scaling Group (cheap, always on)
- Overflow: AWS Lambda (instant, pay per use)
- ALB weighted target groups split the traffic
- Forecast sets instance count (hourly); observed rate sets weights (per minute)

$$\text{base } b^* = F^{-1}(q^*)$$

$$q^* = 1 - f / L$$

f: container $ per unit-minute
L: Lambda $ per unit-minute

0.0013%

gap between the rule and brute-force search (test hours)

Data

Azure Functions 2019 trace, filtered to busy functions

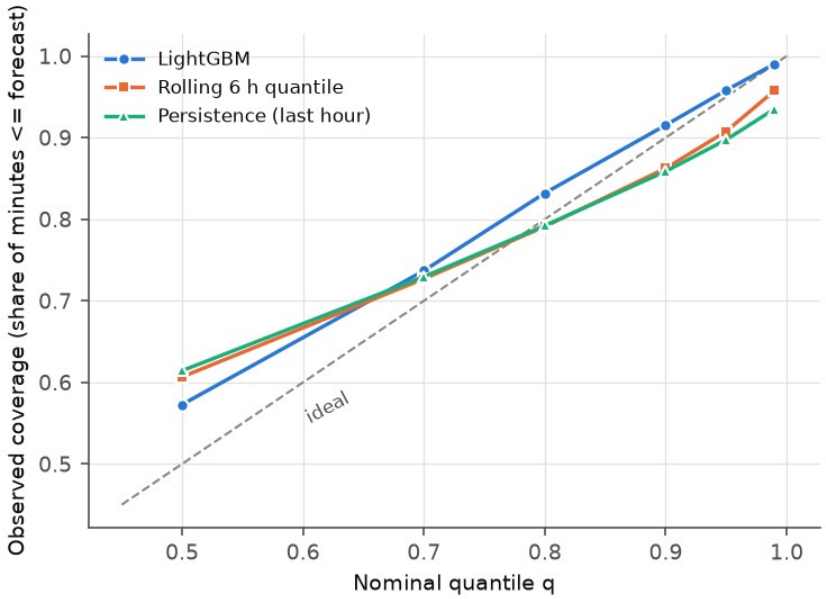
Step	Count
Distinct functions (14 days)	72,359
On day 1	46,412
On all 14 days	28,859
>= 5 req/min, all 14 days	2,127
+ duration + memory	2,066
Heavy (>= $50/mo)	101

- Load = invocations/min ÷ 60 × duration (concurrency)
- Timer functions excluded (periodic, not diurnal)
- Chronological split: train 1-10, validation 11-12, test 13-14
- Azure traffic priced with AWS prices

Features and model

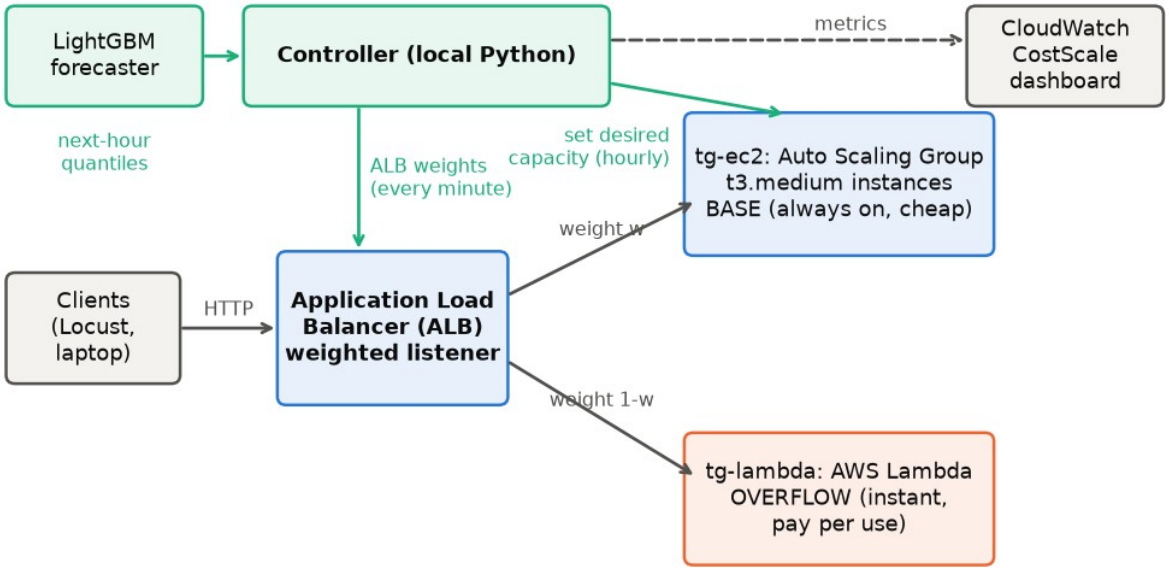
LightGBM quantile regression, next-hour horizon

- Targets: q in {0.5, 0.7, 0.8, 0.9, 0.95, 0.99} of next-hour per-minute concurrency
- Features: last 60 / 360 min (mean, max, q90), same hour yesterday, hour of day, trigger, app level
- Margin multiplier chosen on validation by decision cost
- Baselines: persistence, rolling 6 h, seasonal naive, static



Architecture

Forecast sets instances, observed load sets weights

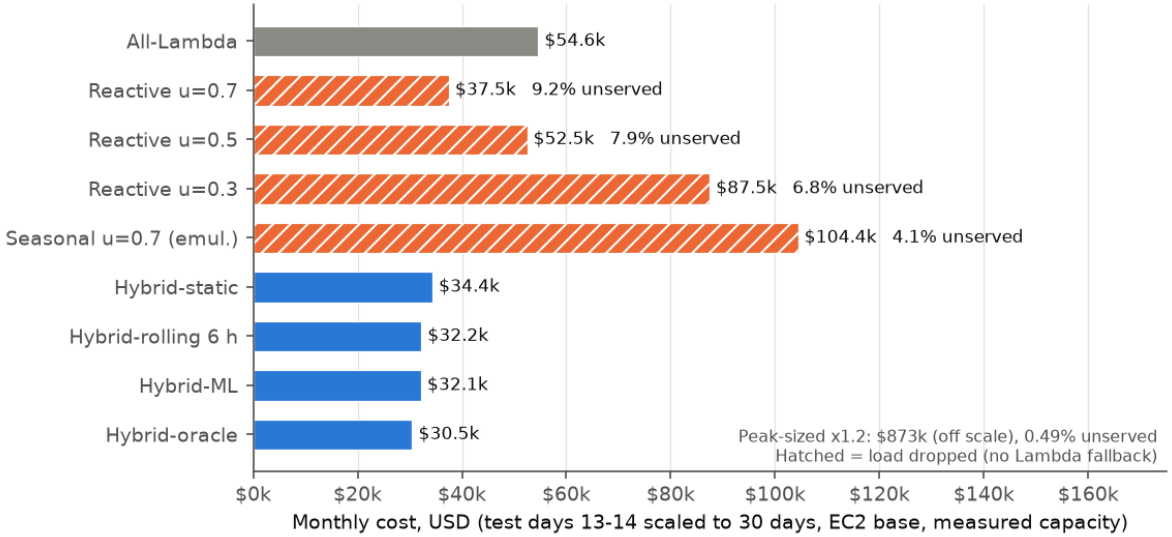


Overflow = load above base capacity. EC2 share w = min(1, base_rps / observed_rps). Region: ap-south-1.

- ALB weights 0-999, per minute
- ASG desired capacity, per hour
- Lambda 1769 MB (1 vCPU) as overflow
- make up / make down, tagged, verified

Simulation results

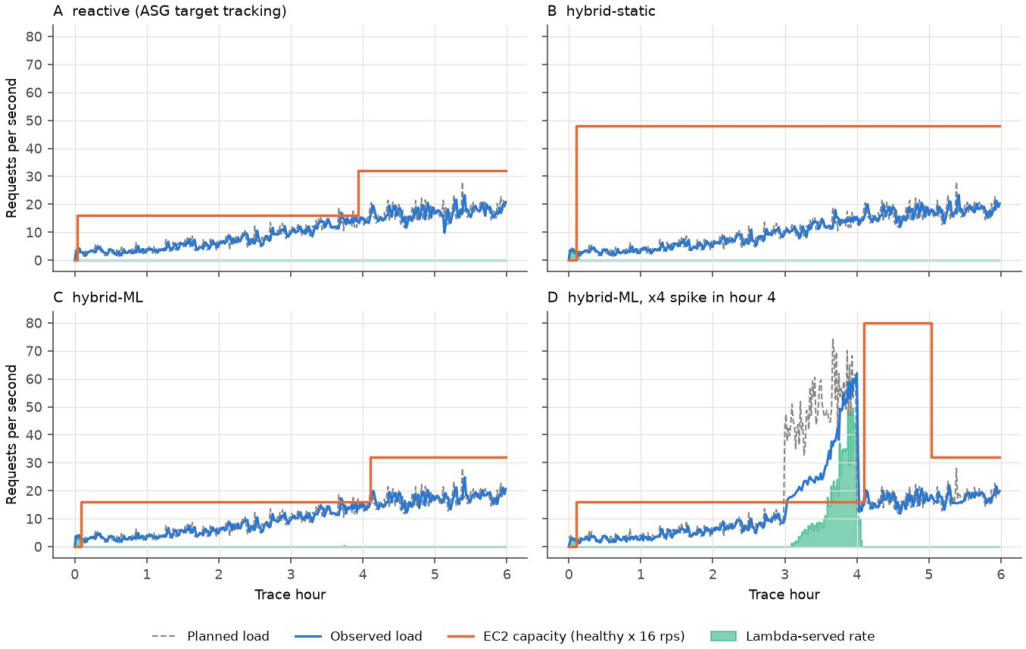
The hybrid wins on cost and unserved load; ML adds almost nothing over rolling



- Hybrid-rolling $32.2k vs all-Lambda $54.6k
- ML vs rolling: -0.25% (noise level)
- Headroom to oracle: 5.4%
- Energy (estimate): hybrid 3,455 kWh vs all-Lambda 1,040 kWh

Live results on AWS

Four arms, same trace, run once each

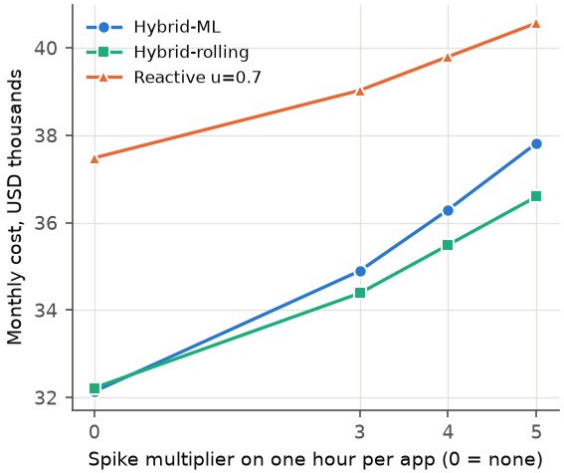


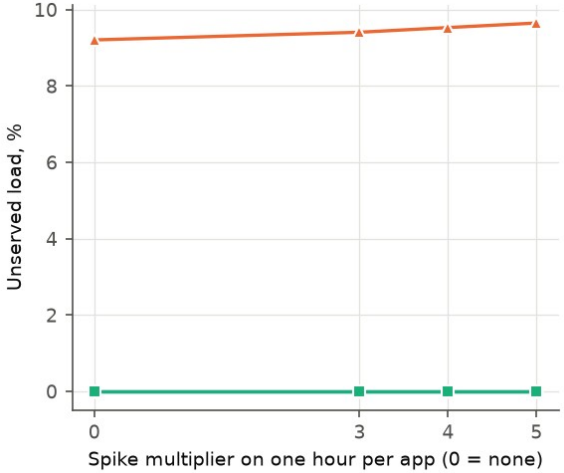
Arm	Errors	Lambda	p95 ms	Cost
A reactive	0.1%	0%	566	$0.038
B static	0.0%	1%	517	$0.085
C ML	0.0%	1%	585	$0.039
D ML + spike	0.0%	20%	2618	$0.124

- A ≈ C in cost on a smooth ramp; static base B costs 2.2x
- Spike (D): 0 failed, but 20% slower than 800 ms
- Replay agrees within ~2% for A-C
- Weight changes take ~12 s; launch 29-31 s

What if the prediction is wrong?

Under-prediction goes to Lambda: nothing fails, but latency can suffer





- Too small base: overflow to Lambda, higher cost, higher latency on cold start
- Too large base: idle containers, wasted money only
- Reactive: under-provisions and drops load
- Live arm D: 0 failed, but 20% of requests slower than 800 ms
- x5 spike (sim): +$4.4k (rolling) / + $5.7k (ML) per month, 0% dropped

Limitations and future work

Honest boundaries

- Azure traffic, AWS prices
- One request type, one instance type, coarse capacity grid
- Short, compressed, single-run live arms
- Energy is an estimate; predictive scaling not run live
- Advantage depends on capacity and Lambda price
- Storage tiering and network cost (out of scope here)
- Fargate base, measured and run live
- Multi-region placement
- Anomaly-aware forecasts
- Overflow-only routing instead of weights

Conclusion

A principled base, an honest result

- Newsvendor rule sets the base; matches brute force
- Hybrid: $32.2k/month, 0% dropped, vs $54.6k all-Lambda and reactive $37.5k with 9.2% dropped
- ML forecaster ≈ rolling quantile (reported plainly)
- Hybrid saves cost, not energy (estimate)
- Live AWS runs: mechanism works; A ≈ C in cost on a smooth ramp; spike hurt latency

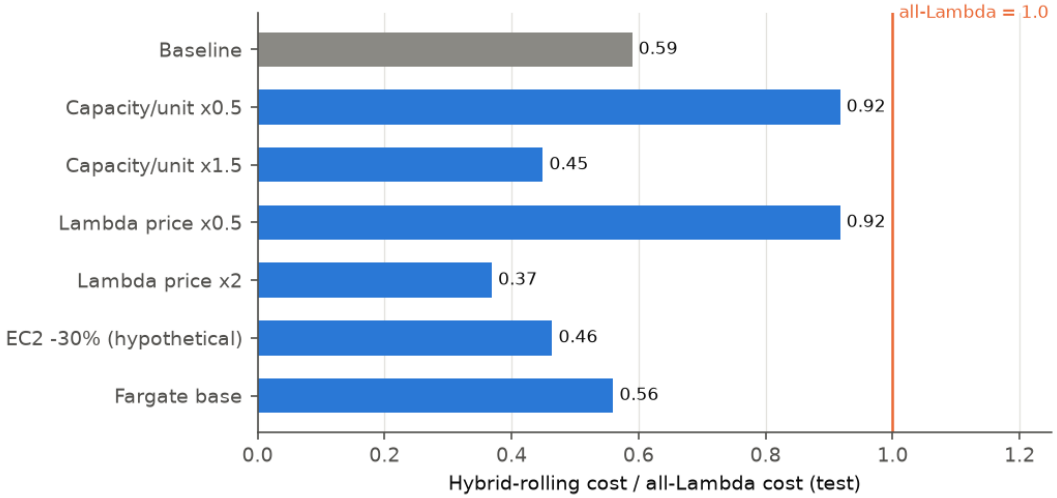
Backup: cost-model assumptions

Backup

Item	Value
Lambda duration	$0.0000167 per GB-s
Lambda requests	$0.0000002 per request
EC2 t3.medium	$0.0448 per hour
Fargate vCPU / GB	$0.04256 / $0.004655 per hour
EC2 capacity unit	6.4 concurrent requests (measured; assumed 16)
Fargate capacity unit	4 concurrent requests (assumed)
Prices retrieved	2026-10-24

Backup: sensitivity

Backup



- Capacity x0.5: saves 8% vs all-Lambda
- Lambda price x0.5: saves 8%
- Assumed capacity 16: $17.0k vs $53.8k